

第一周学习笔记

从 Maven 地基到三节点 Raft 写链
结合真实问答、实验输出与薄弱点重组

学习环境：Windows + IntelliJ IDEA + JDK 11

验证环境：3 台 CentOS 7.x 虚拟机

源码主线：Arithmetic Example · Client · Server · RaftLog · StateMachine

版本：2026-07-24 · 重点补强版

使用说明：这不是流水账，而是一张可复习的知识地图

本笔记的定位
第一周的价值不是记住很多类名，而是建立一套可重复的方法：先固定环境和 commit，再从入口沿调用链阅读，用测试、断点和三节点日志验证，最后通过 Git 把 Windows 修改安全同步到 Linux。

根据你这一周的问答，薄弱点主要集中在以下五类：

问题类型	问题描述	本笔记的解决方式
Maven 心智模型	Reactor、-am、本地仓库、JAR、classpath 容易混在一起	把项目选择、生命周期、依赖解析、运行加载拆成四个阶段
抽象接口与实际对象	RaftClient、BlockingApi、RaftClientRpc 的类型层次容易跳跃	每条链都写清声明类型、实现类型、创建点和首次跨模块调用
时间顺序	append、majority、commit、apply、reply 容易压成一步	用六时刻表和三个索引拆开
Debug 取证	断点影响选举、Tracepoint 不暂停、测试并非一定走 gRPC	先确认测试模型，再选择普通断点、条件断点或 Tracepoint
Git 三棵树	git diff、staged、status --short、同步脚本保护容易混淆	单独设章，用 HEAD/Index/Working Tree 建立统一模型

推荐复习顺序

- 第一次：通读每章的“先记结论”和对照表，不追求记住所有源码位置。
- 第二次：打开 Ratis 源码，对照 Client 和 Server 两条主链逐跳定位。
- 第三次：遮住答案，完成每章末尾的自测问题。
- 遇到构建或同步故障时，直接查命令速查表，不凭记忆反复试参数。

目录

使用说明：这不是流水账，而是一张可复习的知识地图	2
推荐复习顺序	2
目录	3
第 1 章 第一周总览：你真正建立了什么	6
贯穿全周的三条主线	6
第 2 章 环境与构建基线：先消除非业务变量	7
2.1 为什么 Windows 和三台 Linux 都固定 JDK 11	7
2.2 已安装 Maven，为什么仍优先 mvnw	7
2.3 PowerShell 脚本执行策略	7
2.4 Protobuf 标红：生成成功与 IDEA 索引是两件事	7
2.5 CentOS 7 工具链问题	8
第 3 章 Maven 补强：从 POM 到 JVM 加载	9
3.1 一张图建立 Maven 心智模型	9
3.2 生命周期、阶段、插件 Goal	9
3.3 什么是当前 Reactor	9
3.4 为什么 -am 能 package 成功，但 app JAR 没有 library 的 class	10
3.5 本地仓库与 Reactor 不要混为一谈	10
3.6 Maven 如何识别“依赖版本冲突”	10
3.7 为什么编译成功，运行却 NoSuchMethodError	10
3.8 Thin JAR、Fat/Uber JAR、Shade、Relocation	11
3.9 Surefire 参数与证据边界	11
第 4 章 JCommander 与 Runner：命令行如何进入具体业务类	12
4.1 先分清注册对象和注册子命令	12
4.2 为什么 server 后只能解析 ServerCommand 的参数	12
4.3 programName() 与 --help	12
4.4 Runner 的职责	13
第 5 章 Arithmetic 示例全景：不要把所有 Client 混成一个	14
5.1 七个角色	14
5.2 组件关系图	14
5.3 RaftGroup、RaftPeer、RaftProperties	14
5.4 读与写不是同一条 Server 分支	15

第 6 章 同步 Client 链路：接口、实现与首次 RPC	16
6.1 从 Assign 到 gRPC	16
6.2 RaftClientRequest 关键字段	16
6.3 Client 并不是先探测所有节点再发送	16
第 7 章 异步 Client：Future、窗口、有序与重试	18
7.1 异步不等于“绝不阻塞”	18
7.2 主调用链	18
7.3 callId、seqNum、attemptCount	18
7.4 PendingOrderedRequest 与 Future	18
7.5 SlidingWindow 的两端职责	18
第 8 章 Server 写链：把六个时刻彻底拆开	19
8.1 六时刻表	19
8.2 请求入口与 Leader 检查	19
8.3 startTransaction 不是 applyTransaction	19
8.4 一个 Follower 对应一个 LogAppender	19
8.5 三节点多数派	20
8.6 commitIndex 与 appliedIndex	20
8.7 为什么业务日志 index 出现 1、3、5	20
第 9 章 Debug 补强：让断点成为证据，而不是干扰源	21
9.1 先问：当前测试走什么 RPC	21
9.2 普通断点、条件断点、Tracepoint	21
9.3 为什么断点会影响 Leader 选举	21
9.4 为什么总停在 checkLeaderState，走不到 appendLog	21
9.5 IDEA JUnit 配置的意义	22
9.6 断点未命中的排查顺序	22
第 10 章 三节点实验：从现象到 Raft 结论	23
10.1 三台机器的一致性门禁	23
10.2 3/2/1 节点行为	23
10.3 term 增加不等于已经选出 Leader	23
10.4 启动顺序不决定 Leader	23
10.5 Leader 故障与客户端重试	23
10.6 单节点写入为何可能一直等待	23
10.7 节点恢复：原 id、原 storage、自动追赶	24

10.8 FORMAT 与 RECOVER	24
第 11 章 安全修改与测试：先证明测试真的有约束力	25
11.1 createExpression 的解析顺序	25
11.2 @VisibleForTesting	25
11.3 assertThrows 的真实执行顺序	25
11.4 为什么要做两次故意失败实验	25
11.5 最小测试与扩大回归	26
11.6 Day7 证据闭环检查表	26
第 12 章 Git 专章：Windows 修改到三台 Linux 的可证明同步	27
12.1 origin 改名为 study 的含义	27
12.2 本地 branch 与远端 branch 可以不同名	27
12.3 HEAD、短 hash 与完整 hash	27
12.4 Git 三棵树	27
12.5 status --short 的 XY 两列	28
12.6 为什么 git diff 可能“什么都看不到”	28
12.7 精确暂存，不使用 git add	28
12.8 trailing whitespace 与 LF/CRLF	28
12.9 为什么同步脚本拒绝 staged 未提交内容	28
12.10 旧 Git 兼容性	29
12.11 代理与 DNS	29
12.12 提交前最终门禁	29
第 13 章 高频误区与一页复习清单	30
13.1 十组必须能区分的概念	30
13.2 遇到问题时的固定诊断顺序	30
13.3 第一周自测题	30
13.4 推荐的第二周入口	31
附录 A 命令速查	32
Windows 构建与测试	32
Git 检查、提交与同步	32
Linux 环境与构建	32
附录 B 源码索引与笔记来源	33

第 1 章 第一周总览：你真正建立了什么

第一周不是“学完 Raft”，而是建立源码学习闭环。完成闭环后，你应能把同一个现象同时映射到命令、源码、运行时对象和日志证据。

天	主题	知识点/操作
Day 1	Windows 基线	JDK 11、Maven Wrapper、IDEA、全仓构建与工作区保护
Day 2	三台 CentOS 与同步	相同 branch/commit/clean tree, Windows 编辑、Linux 验证
Day 3	Arithmetic 全景	区分 CLI、RaftGroup、Client、Server、RaftLog、StateMachine
Day 4	Client 请求链	同步/异步入口、请求字段、Leader 选择与重试
Day 5	Server 写链	Leader 检查、本地追加、复制、提交、apply、reply
Day 6	三节点实验	3/2/1 节点行为、重新选主、日志追赶和证据边界
Day 7	安全小改动	基线、负向测试、最小回归、diff 自审、精确提交、跨平台验证

贯穿全周的三条主线

主线	从源码开始	从日志开始
构建主线	POM + Reactor + 生命周期	class/JAR/Surefire 报告
写请求主线	Assign 产生业务 Message	Raft 日志提交后 StateMachine apply 并回复
工程主线	Windows 修改和测试	commit -> GitHub -> 三台 Linux 同 commit 复验

最重要的学习习惯
每次回答都尽量写成“源码位置 + 运行时观察 + 我的解释”。只有方法名没有职责，只有 BUILD SUCCESS 没有 Tests run，只有 Client 成功没有 Server 证据，都不构成完整闭环。

第 2 章 环境与构建基线：先消除非业务变量

2.1 为什么 Windows 和三台 Linux 都固定 JDK 11

源码学习最怕把环境差异误判为业务问题。JDK、Maven、编码和 commit 必须先固定。Windows 的 IDEA Project SDK、Maven Runner JRE、命令行 JAVA_HOME 应指向同一套 JDK 11；三台 CentOS 也要记录完整 `java -version`。

Tool	Command	Verification
JDK	<code>java -version</code> 显示 11	不能证明 Maven 实际也使用该 JDK
Maven	<code>./mvnw -version</code> 同时显示 Maven、Java、encoding	不能证明目标测试执行
Git	branch、完整 commit、 <code>status --short</code>	仅 branch 相同不能证明源码相同
IDEA	Project SDK 和 Maven Runner JRE 都是 JDK 11	IDE 绿色不等于命令行构建通过

2.2 已安装 Maven，为什么仍优先 mvnw

Maven Wrapper 固定的是项目预期 Maven 版本和启动方式。你本机安装 `mvn` 并不错误，但团队或多机实验更关心“大家是否真的用了同一版本”。Wrapper 还能降低 PATH 配置差异。

```
# Windows
.\mvnw.cmd -version

# Linux
./mvnw -version
```

不要混淆

Wrapper 固定 Maven 启动版本，不会替你固定 JDK；Maven 输出中的 Java version 才是它实际使用的 JDK。

2.3 PowerShell 脚本执行策略

“禁止运行脚本”是 PowerShell ExecutionPolicy 拦截，不是脚本逻辑错误。一次性执行可使用 Bypass，不必为了学习环境永久降低整个系统策略。

```
powershell -ExecutionPolicy Bypass -File `
learning-plan/week-01/scripts/windows/Test-StudyEnvironment.ps1
```

2.4 Protobuf 标红：生成成功与 IDEA 索引是两件事

- Maven protobuf 插件先把 .proto 生成 Java 到 target/generated-sources。
- 生成目录必须被 Maven 模型识别为 generated source root，不能把整个 target 手工标成普通源码。
- Maven BUILD SUCCESS 但 IDEA 标红，优先检查 Maven Reload、模块归属和索引，而不是编辑生成的 RaftProtos.java。
- 生成文件顶部的 DO NOT EDIT 是硬边界：重新生成时手工修改会丢失。

2.5 CentOS 7 工具链问题

CentOS 7 自带 libstdc++ 较旧，新的 protoc-gen-grpc-java 本地插件可能要求 CXXABI_1.3.8、CXXABI_1.3.9 或 GLIBCXX_3.4.21。此类报错发生在本地二进制加载阶段，不是 Java 源码编译错误。第一周通过在 /usr/local 安装 GCC 14.3 并正确暴露运行库解决。

判断层级

看到 PROTOC FAILED + libstdc++.so.6 version not found，应先查本地原生运行库；看到 Java symbol not found 才进入 Java 依赖或生成源码排查。

第 3 章 Maven 补强：从 POM 到 JVM 加载

3.1 一张图建立 Maven 心智模型

```
pom.xml
|
+-- 项目坐标、modules、dependencies、plugins
|
v
Reactor: 决定 "这一次构建哪些项目、按什么顺序"
|
v
生命周期: validate -> compile -> test -> package -> install
|
v
插件 Goal: 真正执行 javac、surefire、jar、shade 等动作
|
v
target/classes、target/test-classes、JAR、测试报告
|
v
运行时 classpath: JVM 最终从哪里加载类
```

3.2 生命周期、阶段、插件 Goal

概念	例子	注意事项
生命周期阶段	compile、test、package、install	执行后面的阶段会连带执行前面的阶段
插件 Goal	surefire:test、compiler:compile	真正做事的实现单元
系统属性	-DskipTests、-Dtest=...	给插件或项目传参数，不是生命周期阶段

Windows 参数陷阱

PowerShell 对 -D 参数的解析可能造成 Unknown lifecycle phase。将完整属性放在双引号中，例如 "-Dsurefire.failIfNoSpecifiedTests=false"。

3.3 什么是当前 Reactor

Reactor 不是永久存在的仓库对象，而是某次 Maven 命令启动后，根据入口 POM、-pl、-am 等参数计算出来的项目集合和顺序。命令结束，当前 Reactor 也结束。

```
.\mvnw.cmd -pl ratis-examples -am test

# 当前 Reactor 包含:
# 1. ratis-examples
# 2. 它在本次多模块源码树中依赖的上游模块
# 3. 按依赖关系排序
```

-pl 与 -am 的准确含义

参数	含义	常见误解
-pl ratis-examples	选择目标项目	不是“只编译一个目录”
-am	把目标项目依赖的 Reactor 上游项目一起加入	不是把所有第三方依赖重新打包
-amd	把依赖目标项目的下游项目加入	与 -am 方向相反

3.4 为什么 -am 能 package 成功，但 app JAR 没有 library 的 class

-am 解决的是构建顺序和 Reactor 内依赖可用性，不等于把依赖内容合并进 app JAR。编译 app 时，Maven 可以直接把 Reactor 中 library 模块的 target/classes 加入编译 classpath，因此即使本地仓库还没有该坐标，编译仍能成功。普通 app JAR 默认只打包本模块内容。

```
Reactor 编译时:
greeting-app javac classpath
= greeting-library/target/classes + 其他依赖

package 后:
greeting-app.jar
= greeting-app 自己的 class/resources
!= 自动包含 greeting-library.class
```

3.5 本地仓库与 Reactor 不要混为一谈

坐标	何时使用	输出路径
当前 Reactor	同一次多模块构建中直接提供模块输出	module/target/classes
本地仓库	Reactor 外解析已 install/download 的构件	~/.m2/repository
远程仓库	本地没有时下载	Maven Central 或镜像

3.6 Maven 如何识别“依赖版本冲突”

Maven 的依赖仲裁主要根据坐标 groupId + artifactId 判断“同一个依赖”的多个版本，然后应用 nearest definition 等规则选择一个版本。version 用来区分候选版本，但冲突身份主要由 GA 确定。

两个完全不同坐标的 JAR 如果都包含同名 class，Maven 通常不会替你仲裁，因为它们不是同一个依赖坐标。两个 JAR 都可能留在 classpath，最终由 JVM 按 classpath 顺序先找到谁就加载谁。这属于重复类 / classpath 冲突。

坐标	Maven 行为	运行结果
同 GA，不同 version	依赖仲裁后通常保留一个版本	编译版本与运行版本不兼容可触发 NoSuchMethodError
不同 GA，包含同名 class	通常都保留	classpath 顺序决定加载哪个 class
类根本不存在	编译或运行时报找不到类	ClassNotFoundException / NoClassDefFoundError

3.7 为什么编译成功，运行却 NoSuchMethodError

```
consumer-a 编译时 -> third-party v1: 方法存在
consumer-b 编译时 -> third-party v2: 方法存在

最终运行 classpath -> 只能先加载一个同名 third-party class
如果实际加载版本没有调用方法字节码期待的方法:
java.lang.NoSuchMethodError
```

编译器只验证当时的编译 classpath。运行时由 JVM 根据实际 classpath 重新加载类，所以 BUILD SUCCESS 不能证明运行期二进制一定兼容。

3.8 Thin JAR、Fat/Uber JAR、Shade、Relocation

类型	描述	是否自动包含依赖
普通/Thin JAR	当前模块 class 和 resources	否
Fat/Uber JAR	把依赖内容也合并进一个 JAR	是
Shade	“遮蔽/着色”，构建 fat JAR，并可改写依赖	通常是
Relocation	把依赖包名及字节码引用改到新命名空间	解决类身份冲突

Ratis ThirdParty 的核心不是“单纯把依赖打大包”，而是通过 relocation 隔离依赖命名空间，降低 Ratis 内部依赖与用户应用依赖冲突。skipShade 通常用于跳过耗时的 shade 阶段，但跳过后得到的产物语义可能不同，不能把它当作发布物验证。

3.9 Surefire 参数与证据边界

```
.\mvnw.cmd -pl ratis-examples -am `
"-DskipTests=false" `
"-Dtest=TestAssignCli" `
"-Dsurefire.failIfNoSpecifiedTests=false" `
test
```

输出证据	证据含义
-DskipTests=false	确保测试执行；不要只写成“编译测试代码”
-Dtest=TestAssignCli	指定目标测试类
failIfNoSpecifiedTests=false	上游模块没有该测试时继续，不忽略真实失败
Running TestAssignCli	证明目标测试类被发现
Tests run: 2, Failures: 0	证明执行数量和结果
BUILD SUCCESS	整个 Reactor 成功，但单独看它证据不足

必须记住

测试名拼错时，failIfNoSpecifiedTests=false 可能让整个构建仍然绿色。因此至少同时检查 Running 和 Tests run。

第 4 章 JCommander 与 Runner：命令行如何进入具体业务类

4.1 先分清注册对象和注册子命令

方法	解析规则	使用场景
<code>addObject(object)</code>	没有额外子命令名，直接解析该对象参数	单命令程序、全局参数
<code>addCommand("server", object)</code>	先匹配 <code>server</code> ，再解析该对象参数	一个程序包含多个子命令

```
JCommander jc = JCommander.newBuilder()
    .programName("arithmetic")
    .addObject(globalOptions)
    .addCommand("server", serverCommand)
    .addCommand("assign", assignCommand)
    .build();
```

`addObject` 与 `addCommand` 可以同时存在。典型设计是：`addObject` 注册全局参数，`addCommand` 注册各子命令参数。它们不是互斥关系，关键在于每个参数属于哪个命名空间。

4.2 为什么 `server` 后只能解析 `ServerCommand` 的参数

```
arithmetic server --id n0 --port 7000 --name testName
      ^^^^^^
      commands Map 中的 key

解析器先进入 server 子命令空间，
随后只在 ServerCommand 及其继承字段中匹配参数。
```

如果 `--name` 只声明在 `AssignCommand`，那么进入 `server` 命名空间后不会去其他子命令对象查找，所以会报 `parameter error`。继承自 `SubCommandBase` 的公共参数之所以可用，是因为它们本来就是 `ServerCommand` 对象字段的一部分。

4.3 `programName()` 与 `--help`

- `programName()` 主要影响 `usage/help` 输出中显示的程序名，不参与参数值绑定。
- `JCommander` 不会凭空替你的业务程序完成所有 `--help` 行为；通常需要声明 `help=true` 的 `@Parameter`，并在 `parse` 后调用 `usage()`。
- `help=true` 的意义是：出现 `--help` 时跳过 `required` 参数校验，让程序有机会打印帮助。

```

@Parameter(names = "--help", help = true)
private boolean help;

JCommander jc = JCommander.newBuilder()
    .programName("demo")
    .addObject(options)
    .build();
jc.parse(args);
if (options.help) {
    jc.usage();
    return;
}

```

4.4 Runner 的职责

Runner 是命令行入口和调度器。它不实现 Raft 写入，而是完成四件事：创建/注册命令对象，解析 args，找出 parsedCommand，处理 usage/error，然后调用匹配对象的 run()。

```

main(args)
-> 构建 JCommander
-> addCommand(name, commandObject)
-> parse(args)
-> getParsedCommand()
-> commands.get(name)
-> command.run()
-> Client.run() / Server.run()
-> 进入 Ratis API

```

阅读技巧

看到抽象父类时，不要只问“它实现了什么”，还要问：哪些字段由父类统一提供，哪些参数由子类补充，Runner 最终持有和调用的是哪个具体对象。

第 5 章 Arithmetic 示例全景：不要把所有 Client 混成一个

5.1 七个角色

角色	职责	备注
Assign/Get	解析 CLI 业务参数，构造 Message	不是网络 Client 实现
RaftClient	客户端 API 抽象	变量声明类型，不代表实际实现类
RaftClientImpl	Leader 选择、请求构造、同步/异步 API	不执行状态机业务
RaftClientRpc	客户端 RPC 抽象	具体实现可为 gRPC 或 simulation
RaftServerImpl	Server 请求分派和写流程	不是 RaftServerProxy
RaftLog	持久化有序日志及 commitIndex	不是业务 variables
ArithmeticStateMachine	真正计算并保存变量状态	只应用 committed 日志

5.2 组件关系图



5.3 RaftGroup、RaftPeer、RaftProperties

- RaftPeer 描述一个成员：id、address、priority 等。
- RaftGroup 把 groupId 和 peers 组合成一个 Raft 组。
- RaftProperties 保存 Ratis 配置，不是集群成员列表。
- Client 和 Server 必须对 groupId、peer id/address 形成一致认识。

5.4 读与写不是同一条 Server 分支

操作	读写路径/日志流	是否写入 Raft 日志
assign	WRITE -> writeAsync	是，需要 Leader 建序、复制、提交、apply
get	READ_ONLY/查询路径	通常直接查询状态机，不产生同类业务日志

第 6 章 同步 Client 链路：接口、实现与首次 RPC

6.1 从 Assign 到 gRPC

```
Assign#operation
-> client.io()
-> BlockingImpl#send(Message)
-> BlockingImpl#send(Type, Message, RaftPeerId)
-> BlockingImpl#sendRequest(...)
-> RaftClientImpl#newRaftClientRequest(...)
-> client.getClientRpc().sendRequest(request)
-> GrpcClientRpc#sendRequest(...)
-> GrpcClientProtocolClient
```

问题	答案
client 声明类型	RaftClient
client 实际类型	RaftClientImpl
io() 声明返回类型	RaftClientRpc.BlockingApi
io() 实际对象	BlockingImpl
RaftClientRpc 实际实现	Arithmetic CLI 中显式设置为 GrpcClientRpc
第一次调用 RPC 抽象	BlockingImpl 发送请求时调用 client.getClientRpc()

6.2 RaftClientRequest 关键字段

字段	作用	需要维护吗
clientId	客户端身份	某个 Server id
groupId	目标 Raft 组	Git branch 或进程组
callId	标识客户端逻辑请求，重试通常保持	重试次数
serverId	本次目标节点/当前认为的 Leader	永远正确的 Leader
message	Arithmetic 业务消息	Builder 自动生成的业务数据
type	WRITE、READ 等请求语义	Java 类类型
attemptCount	本逻辑请求已尝试次数	callId

6.3 Client 并不是先探测所有节点再发送

Client 初始化时会从显式 leader、缓存或 peers 中选择一个初始目标。该目标只是“当前认为的 Leader”。如果发错节点，Server 返回带 NotLeaderException 的 reply，Client 使用 suggestedLeader/peers 更新信息，再由 RetryPolicy 决定是否等待和重试。

初始目标

- > 发送请求
- > Server#checkLeaderState()
- > NotLeaderException(suggestedLeader, peers)
- > Client 更新 leaderId / peers
- > RetryPolicy -> RetryAction
- > 同一 callId, attemptCount 增加, 目标可能改变

证据边界

Client 最终 Success 不代表第一次就发给 Leader。中间 NotLeaderException 可能被客户端内部吸收；
需要结合目标 Server 日志或 Debug 才能证明重试过程。

第 7 章 异步 Client: Future、窗口、有序与重试

7.1 异步不等于“绝不阻塞”

`async().send()` 返回 `CompletableFuture`, 表示结果稍后完成; 调用者可 `thenAccept` 注册动作, 也可 `join` 主动等待。但入口可能在 `requestSemaphore.acquire()` 等待并发额度, 所以“异步 API 任何情况下都不阻塞调用线程”是不准确的。

7.2 主调用链

```
RaftClientImpl#async
-> AsyncImpl#send
-> RaftClientImpl#getOrderedAsync
-> OrderedAsync#send
-> SlidingWindow.Client#submitNewRequest
-> OrderedAsync#sendRequestWithRetry
-> GrpcClientRpc#sendRequestAsync
```

7.3 callId、seqNum、attemptCount

字段	主要用途	是否保持
callId	标识同一个客户端逻辑请求, 支持 retry cache 去重	保持
seqNum	标识同一客户端滑动窗口中的顺序	保持
attemptCount	记录该请求第几次尝试, 供重试策略使用	增加
target server	本次发送目标	Leader 变化时可能改变

已经有 callId 仍需要 seqNum, 因为“唯一身份”和“窗口顺序”是两个维度。callId 回答“是不是同一个逻辑请求”, seqNum 回答“它在这个客户端的有序请求流中排第几”。

7.4 PendingOrderedRequest 与 Future

- Pending 表示请求已经进入客户端管理, 但最终 reply 尚未完成。
- 对象保存 callId、seqNum、request constructor、replyFuture 等。
- RPC 回复即使乱序到达, 也可通过 seqNum/窗口状态关联到正确 Future。
- 有序异步允许多个请求同时在途, 不等于等前一个完成才发送下一个。

7.5 SlidingWindow 的两端职责

位置	职责
Client window	分配 seqNum、控制 first request、并发额度、重试和 Future 完成顺序
Server window	识别窗口起点, 缓存乱序请求, 按连续 seqNum 处理/回复
Raft log	提供集群范围的一致日志顺序; 不能被客户端窗口替代

容易说错

“回复先到”不等于业务请求可以越过前序执行; “有序异步”也不等于系统中只有一个在途请求。

第 8 章 Server 写链：把六个时刻彻底拆开

8.1 六时刻表

时刻	发生了什么	此时还不能做什么
T1 接收	RaftServerProxy 按 groupId 找 Division	尚未产生日志 term/index
T2 校验	RaftServerImpl 检查 Leader, startTransaction	尚未修改业务状态
T3 本地 append	Leader 为日志分配 term/index 并持久化	尚未形成多数派
T4 复制	每个 LogAppender 面向一个 Follower	不要求三台全部完成
T5 commit	多数派 match 位置允许推进 commitIndex	appliedIndex 可能仍落后
T6 apply/reply	StateMachineUpdater 调用 applyTransaction, 完成业务结果	Follower 不必都已 apply

8.2 请求入口与 Leader 检查

```
RaftServerProxy#submitClientRequestAsync  
-> RaftServerImpl#submitClientRequestAsync  
-> submitClientRequestAsyncInternal  
-> replyFuture  
-> writeAsync  
-> writeAsyncImpl  
-> checkLeaderState
```

非 Leader 不能“先追加再说”。只有 Leader 能为 Client 写请求建立唯一日志顺序。Follower 直接接收 Client 写入会造成同 index 不同内容，破坏一致性。因此 checkLeaderState 是写链的早期门禁。

8.3 startTransaction 不是 applyTransaction

方法	输入输出	描述
startTransaction	RaftClientRequest -> TransactionContext	校验、解析、准备日志和状态机上下文
appendTransaction	TransactionContext -> 日志追加 Future	分配 term/index, 追加 Leader 本地日志
applyTransaction	committed TransactionContext -> Message Future	真正修改 Arithmetic variables

8.4 一个 Follower 对应一个 LogAppender

Leader 不是用一个 LogAppender 同时管理所有 Follower。每个 Follower 通常对应一个发送器状态，维护 nextIndex/matchIndex 等进度。某个 Follower 落后时，Leader 可以单独回退 nextIndex 并补日志。

8.5 三节点多数派

```
N = 3
majority = floor(N / 2) + 1 = 2

Leader 本地 + 任意 1 个 Follower = 2 -> 可形成多数派
只有 Leader 自己 = 1 -> 不可提交新日志
```

8.6 commitIndex 与 appliedIndex

lastLogIndex	本地已经拥有的最后日志位置	可能包含未提交日志
commitIndex	已被多数派确认、允许应用到状态机的位置	不大于 lastLogIndex
appliedIndex	状态机已经实际执行到的位置	通常不大于 commitIndex

```
appliedIndex <= commitIndex <= lastLogIndex
```

Client 成功的证明边界

本例普通写请求成功，可证明 Leader 已满足提交条件并执行状态机产生结果；不能证明两个 Follower 都已 apply，也不能证明三台 appliedIndex 瞬间相同。

8.7 为什么业务日志 index 出现 1、3、5

ArithmeticStateMachine 的 INFO 只打印状态机业务日志，所以看到 a、b、c 位于 1、3、5，并不代表 RaftLog 没有 2、4。当前配置启用日志 metadata 后，Leader 在 commitIndex 推进时会追加 MetadataEntry 记录提交位置。

```
index 1 StateMachineLogEntry: a = 3
index 2 MetadataEntry: 记录新的 commitIndex
index 3 StateMachineLogEntry: b = 4
index 4 MetadataEntry: 记录新的 commitIndex
index 5 StateMachineLogEntry: c = a + b
```

源码证据链：LeaderStateImpl 更新 commit 后调用 logMetadata(lastCommitIndex)，随后 RaftLog.appendMetadata；LogProtoUtils.toLogEntryProto(commitIndex, term, index) 构造 MetadataEntry。ArithmeticStateMachine.applyTransaction 只处理并打印 StateMachineLogEntry，所以日志显示为奇数业务 index。

第 9 章 Debug

补强：让断点成为证据，而不是干扰源

9.1 先问：当前测试走什么 RPC

TestArithmetic 使用 MiniRaftCluster 时，你观察到的 clientRpc 可能是 SimulatedClientRpc，而不是 GrpcClientRpc；因此不能用它证明 GrpcLogAppender 链。测试名相同也不代表底层传输模型相同。先在变量窗口检查实际对象类型，再决定断点位置。

9.2 普通断点、条件断点、Tracepoint

类型	是否暂停	适用场景
普通断点	是	第一次理解局部变量和调用栈
条件断点	条件成立才暂停	过滤特定 serverId、index、request type
Tracepoint	否	高频并发路径、选举/心跳/复制链取证

Tracepoint 中 Evaluate and log 会把表达式结果打印到 Debug Console，然后程序继续运行。因此测试直接结束并不表示 Tracepoint 无效；应查看 Console 是否输出了设置的内容。

9.3 为什么断点会影响 Leader 选举

Raft 依赖心跳和选举超时。普通断点暂停某个关键线程甚至整个 JVM 时，Follower 可能收不到心跳并发起新选举；Leader 也可能因停顿错过时序。频繁 Resume 后角色和 term 已变化，原先计划的链路可能不再成立。

- 第一轮只设 1-3 个断点，不要在心跳高频方法广泛停住。
- 复制和选举路径优先 Tracepoint。
- 若必须暂停，选择 Suspend: Thread 而不是 All，并明确其风险。
- 每轮 Debug 重新启动测试，避免把被干扰后的集群当作干净证据。

9.4 为什么总停在 checkLeaderState，走不到 appendLog

常见原因不是 appendLog 不执行，而是断点命中了多个 Server/Follower 的同名路径。某次请求进入非 Leader 后会提前返回；或者暂停已经触发重新选主。应在 Variables 中检查当前 Division 的 member id/role，并加条件只停目标 Leader。

建议观察顺序：

1. checkLeaderState: 确认当前对象和角色
2. appendTransaction / state.appendLog: 确认 term/index 分配
3. Leader reply handler: 观察 matchIndex/commit 推进
4. StateMachineUpdater: 观察 committed -> applied
5. ArithmeticStateMachine.applyTransaction: 观察业务结果

9.5 IDEA JUnit 配置的意义

固定 Run/Debug Configuration 是为了稳定 module classpath、测试类、JDK、工作目录和 VM 参数。断点只是“在哪里暂停”，配置决定“启动哪个程序、用什么 classpath 跑”。没有正确配置时，可能出现 TestArithmetic ClassNotFoundException，断点再正确也无法开始。

配置项	TestArithmetic 配置
Module / classpath	ratis-examples, 而不是 ratis-test
Test kind	Class
Class	org.apache.ratis.examples.arithmetic.TestArithmetic
JRE	Project JDK 11
Working directory	模块或项目默认目录, 保持 Maven 模型一致

9.6 断点未命中的排查顺序

- 1 测试类是否真的运行：看 Running 和 Tests run。
- 2 当前 classpath 是否包含目标模块 test-classes。
- 3 断点所在方法是否属于当前 RPC/测试实现。
- 4 实际对象类型是否与预期一致。
- 5 条件断点表达式是否错误。
- 6 代码是否重新编译，IDEA 是否使用旧 class。
- 7 最后才考虑业务分支没有进入。

第 10 章 三节点实验：从现象到 Raft 结论

10.1 三台机器的一致性门禁

```
三台都检查：
git branch --show-current
git rev-parse HEAD
git status --short
./mvnw -version
ss -ntlp | grep 6000
```

分支名相同而 commit 不同，不算源码一致；commit 相同但工作树非空，也不能保证实际运行源码一致。必须同时记录 branch、完整 commit、clean tree。

10.2 3/2/1 节点行为

节点数	能否形成多数派 Leader	能否形成多数派	票数
3/3	能	能	多数派 2
2/3	能	能	仍有多数派 2
1/3	不能稳定形成多数派 Leader	不能	只有 1 票

10.3 term 增加不等于已经选出 Leader

正式 Election 进入新 term，但只有获得多数票才成为 Leader。日志出现更高 term 只能证明节点进入了新一轮正式选举或接受了更高 term，不能单独证明 Leader 已产生。还要找 CANDIDATE -> LEADER、Leader ready 或其他节点确认 Leader 的证据。

10.4 启动顺序不决定 Leader

Leader 由选举超时、日志新旧程度、term 和多数票共同决定。先启动的进程可能先成为 Follower，也可能在其他节点加入后才完成选举。日志中出现 leader 单词，也可能只是 Follower 在记录“Leader 从 null 更新为 n0”。

10.5 Leader 故障与客户端重试

```
旧 Leader n0 停止
-> n1/n2 仍构成多数派
-> 新一轮选举，term 增加
-> 新 Leader ready
-> Client 可能先发错节点
-> NotLeaderException(suggestedLeader)
-> 切换目标，写入成功
```

10.6 单节点写入为何可能一直等待

进程存活不等于集群可用。唯一节点可能不断 PreVote/Election、请求其他节点并重试，但无法获得多数票。Client 超时仅证明观察窗口内未成功，不等于进程必然崩溃。失败尝试可能短暂出现在某节点未提交日志中，但不能当成 committed 业务状态。

10.7 节点恢复：原 id、原 storage、自动追赶

- 使用原 id，因为集群配置仍把它识别为同一个成员。
- 使用原 storage，因为其中保存该成员的 term、vote、日志和快照状态。
- 不要复制 Leader storage；Leader 通过 AppendEntries 的 nextIndex/matchIndex 协议让 Follower 自动追赶。
- 出现 previous log entry not found / INCONSISTENCY 时，Leader 会回退 nextIndex，寻找共同前缀。
- 若缺口已被 purge 且存在 snapshot，才可能转为安装 snapshot。

10.8 FORMAT 与 RECOVER

Arithmetic Server builder 默认 FORMAT 时，已有 storage 目录会拒绝启动，以防误格式化。节点恢复必须使用 RECOVER；若 storage 为空，RECOVER 可在适用条件下初始化。恢复实验中不要删除 storage 来“绕过”错误，否则你破坏了最想观察的日志追赶证据。

查询成功的证明边界

Client get 成功只能证明当前服务路径和 Leader 状态机能返回结果，不能单独证明所有 Follower 已追赶完成。必须查 matchIndex、commitIndex、appliedIndex 或节点日志。

第 11 章 安全修改与测试：先证明测试真的有约束力

11.1 createExpression 的解析顺序

```
createExpression(val)
1. NUMBER_PATTERN.matches() -> DoubleValue
2. Variable.PATTERN.matches() -> Variable
3. binaryMatcher.matches() -> 递归解析左右操作数
4. unaryMatcher.matches() -> 递归解析单操作数
5. 否则 throw IllegalArgumentException
```

Matcher.matches() 要求整个字符串匹配，不是只要包含一段即可。输入 `a++b` 既不是完整数字/变量，也无法匹配一个合法二元或一元表达式，所以进入非法分支。

11.2 @VisibleForTesting

`createExpression` 没有显式修饰符，因此是 `package-private`；`TestAssignCli` 与 `Assign` 同包，可以直接调用。`@VisibleForTesting` 不改变 Java 权限，只说明该可见性主要为测试而放宽，避免被误认为稳定包内 API。删除注解后，当前访问权限和运行行为不变，但设计意图丢失。

11.3 assertThrows 的真实执行顺序

```
final IllegalArgumentException exception =
    Assertions.assertThrows(
        IllegalArgumentException.class,
        () -> new Assign().createExpression("a++b"));

Assertions.assertTrue(
    exception.getMessage().contains("Invalid expression"));
```

- 1 JUnit 发现并调用 `@Test` 方法。
- 2 测试方法调用 `assertThrows`，并把 Lambda 作为“稍后执行的动作”传入。
- 3 `assertThrows` 在内部执行 Lambda。
- 4 `createExpression` 抛出异常。
- 5 `assertThrows` 验证类型兼容，并返回异常对象。
- 6 消息断言继续检查业务关键词。

11.4 为什么要做两次故意失败实验

断言类型	断言内容	断言结果
类型断言实验	把 <code>a++b</code> 改为合法 <code>a+b</code>	没有异常时 <code>assertThrows</code> 必须失败
消息断言实验	把关键词改为错误文本	消息断言确实执行且能识别错误消息

绿色 -> 故意变红 -> 恢复绿色，比只看到一次绿色多证明了“测试本身具有灵敏度”。这不是为了制造错误，而是在测试测试。

11.5 最小测试与扩大回归

- 先跑 TestAssignCli: 反馈快, 精确覆盖解析器的负向行为。
- 再跑 TestArithmetic: 检查更大的 Arithmetic 集成路径。
- TestArithmetic 通过不能替代 TestAssignCli 的精确异常断言; 两者证明范围不同。

11.6 Day7 证据闭环检查表

证据	证据闭环检查表	证据闭环
修改前基线	Running + Tests run: 1 + BUILD SUCCESS	只记 BUILD SUCCESS
新增类型断言	Tests run 从 1 变 2	忘记 @Test 仍以为新方法执行
故意合法输入	Failures: 1 + nothing was thrown	只看最终绿色
恢复非法输入	Tests run: 2, Failures: 0	临时 a+b 未恢复
错误消息关键词	消息断言失败	无法证明消息断言有约束力
扩大回归	TestAssignCli + TestArithmetic	用集成测试替代精确单测
Git 自审	diff、cached diff、diff --check、status	只运行普通 git diff

判断完成的标准

代码正确只是第一层; 测试被发现、故意失败有效、临时修改已恢复、提交范围可解释, 四层证据同时成立才算完成。

第 12 章 Git 专章：Windows 修改到三台 Linux 的可证明同步

12.1 origin 改名为 study 的含义

git remote rename origin study 只把远端别名从 origin 改成 study，不会改变 GitHub 仓库内容，也不会把本地 branch 改名。这样做是为了明确：这个远端是个人学习仓库，而不是上游 Apache 仓库。

```
git remote -v
git config --get remote.study.url

# 老版本 Git 不支持 remote get-url 时，
# 使用 git config --get 更兼容。
```

12.2 本地 branch 与远端 branch 可以不同名

```
本地: study
远端: study/week-01

git push -u study study:study/week-01
```

两者可以不同，只要 upstream 映射正确。但当前学习流程最终统一使用本地 study/week-01 和远端 study/week-01，更直观。GitHub 页面显示远端 branch 名，不代表本地当前 branch。

12.3 HEAD、短 hash 与完整 hash

IDE 或命令行可能显示短 hash，例如 737 或 0124e1a1；笔记中的跨机对照应填写 git rev-parse HEAD 的完整 40 位 commit。短 hash 只是完整 hash 的可读前缀。

12.4 Git 三棵树

```
HEAD (上一次提交)
^
| git diff --cached / --staged
|
Index (暂存区, git add 后)
^
| git diff
|
Working Tree (正在编辑的文件)
```

命令	对比范围	用途
git diff	Working Tree vs Index	看尚未 staged 的内容
git diff --cached	Index vs HEAD	看下一次 commit 将包含什么
git diff HEAD	Working Tree + Index vs HEAD	看全部未提交变化
git status --short	汇总两侧状态	快速识别 staged/unstaged/untracked

12.5 status --short 的 XY 两列

XY	含义
M file	工作区已修改，尚未 staged
M file	修改已经 staged
MM file	已 staged 后又继续修改
A file	新文件已 staged
?? file	未跟踪文件
D / D	删除未 staged / 删除已 staged

12.6 为什么 git diff 可能“什么都看不到”

执行 git add 后，Working Tree 与 Index 相同，所以普通 git diff 为空；修改并没有消失，而是已经进入 Index。此时必须使用 git diff --cached 查看即将提交的内容。

12.7 精确暂存，不使用 git add .

```
git add `
  ratis-examples/src/test/java/.../TestAssignCli.java `
  learning-plan/week-01/notes/day-07.md

git diff --cached --name-only
git diff --cached
git diff --cached --check
```

精确暂存能避免把 day-06 笔记、IDE 文件、临时日志等无关修改带入 Day7 commit。提交范围越清楚，Linux 复验和后续回滚越容易。

12.8 trailing whitespace 与 LF/CRLF

问题	原因	解决
trailing whitespace	真实空白检查问题	删除行尾空格，重新 diff --check
LF will be replaced by CRLF	Windows 换行转换 warning	通常不是本次检查失败原因

复制 Maven 日志到 Markdown 时，[INFO] 后常带不可见行尾空格。IDEA 可用正则 [\t]+\$ 替换为空；它只删除行尾空白，不删除正常缩进。

12.9 为什么同步脚本拒绝 staged 未提交内容

Index 只存在于当前 Windows 仓库，Linux 无法 fetch 一个尚未形成 commit 的暂存区。同步脚本要求 clean tree，是为了保证“Windows 运行的 commit”和“Linux 拉取的 commit”可以精确比较。

```
Windows 修改
-> 最小测试
-> git diff / diff --cached / diff --check
-> git add 精确文件
-> git commit
-> Sync-StudyBranch.ps1
-> GitHub study/week-01
-> Linux fetch + checkout/reset 到同一 commit
-> Linux 复验
```

12.10 旧 Git 兼容性

CentOS 7 可能自带老 Git，不支持 `git -C` 或 `git remote get-url`。脚本需要使用 `cd` 后执行 Git，或用 `git config --get remote.study.url` 获取地址。功能缺失不代表远端配置错误。

12.11 代理与 DNS

Could not resolve host: github.com 首先是 DNS/网络解析失败。/etc/profile 的全局代理可能影响 Git，但要分别检查 `http_proxy/https_proxy`、`git config --global http.proxy` 和 DNS。代理配置存在不等于一定是原因，应在节点上用 `getent hosts github.com`、`env` 和 `git config` 分层验证。

12.12 提交前最终门禁

```
git status --short
git diff
git diff --cached --name-only
git diff --cached
git diff --cached --check

# 测试证据
Running: 目标测试类
Tests run: 预期数量
Failures: 0, Errors: 0
BUILD SUCCESS

# 提交后
git rev-parse HEAD
git status --short # 应为空
```

第 13 章 高频误区与一页复习清单

13.1 十组必须能区分的概念

概念	本地仓库	本次多模块构建项目集合 vs 已安装/下载构件缓存
Reactor	本地仓库	本次多模块构建项目集合 vs 已安装/下载构件缓存
-am	fat JAR	把上游加入构建 vs 把依赖内容合入产物
Maven 仲裁	重复 class	同 GA 多版本选择 vs 不同坐标同名类的 classpath 问题
RaftClient	RaftClientImpl	API 声明类型 vs 实际实现对象
callId	seqNum	请求身份 vs 客户端窗口顺序
append	commit	本地拥有日志 vs 多数派确认
commitIndex	appliedIndex	允许应用到哪里 vs 已执行到哪里
Future 返回	业务完成	异步句柄已返回 vs Server 已处理完成
Tracepoint	普通断点	记录并继续 vs 暂停执行
git diff	git diff --cached	未暂存差异 vs 已暂存差异

13.2 遇到问题时的固定诊断顺序

- 1 先判断层级：PowerShell、Git、Maven、Java 编译、测试、RPC、Raft、状态机。
- 2 记录完整命令、完整 commit 和第一处 ERROR，不只截最后一行。
- 3 确认当前运行的实际对象类型和测试模型。
- 4 先做最小可复现实验，再扩大到全链路。
- 5 区分直接证据与推断：看到了 apply，可以推断此前提交；但不能反推每个 Follower 的精确 matchIndex。
- 6 恢复临时改动，运行 diff 自审，再提交同步。

13.3 第一周自测题

- 为什么 -am 构建成功而 app JAR 中没有 library class？
- 两个不同坐标 JAR 含同名 class 时，Maven 为什么不一定仲裁？
- 为什么 Client 的 serverId 不保证第一次就是 Leader？
- callId、seqNum、attemptCount 分别解决什么问题？
- 为什么 Leader 本地 append 不能直接回复成功？
- 三节点中为什么两节点可写、单节点不可写？
- 为什么业务 apply 日志显示 1、3、5，而中间 index 仍可能存在？
- 为什么 Tracepoint 不暂停仍然是正确行为？
- 为什么 TestArithmetic 不一定走 GrpcLogAppender？

- 为什么普通 git diff 为空仍可能存在 staged 修改?

13.4 推荐的第二周入口

序号	主题	目标
1	LeaderStateImpl + LogAppender	把 matchIndex、nextIndex、commit 推进串成一条可调试链
2	StateMachineUpdater	理解 committed、applied、snapshot/purge 的关系
3	RetryCache + SlidingWindow	把同步重试、异步有序和 Server 去重连接起来
4	真实 gRPC 三节点调试	区分 simulation 与 gRPC，建立跨进程日志关联方法

附录 A 命令速查

Windows 构建与测试

```
# 环境
.\mvnw.cmd -version
git status --short

# 快速打包 (不执行测试)
.\mvnw.cmd -DskipTests -Dcheckstyle.skip -Drat.skip clean package

# Arithmetic 集成测试
.\mvnw.cmd -pl ratis-examples -am `
"-DskipTests=false" `
"-Dtest=TestArithmetic" `
"-Dsurefire.failIfNoSpecifiedTests=false" `
test

# TestAssignCli
.\mvnw.cmd -pl ratis-examples -am `
"-DskipTests=false" `
"-Dtest=TestAssignCli" `
"-Dsurefire.failIfNoSpecifiedTests=false" `
test
```

Git 检查、提交与同步

```
git branch --show-current
git rev-parse HEAD
git status --short

git diff
git diff --cached --name-only
git diff --cached
git diff --cached --check

git commit -m "test(examples): cover invalid arithmetic expression"

powershell -ExecutionPolicy Bypass -File `
learning-plan/week-01/scripts/windows/Sync-StudyBranch.ps1
```

Linux 环境与构建

```
git branch
git rev-parse HEAD
git status --short
git config --get remote.study.url

./mvnw -version

./mvnw -pl ratis-examples -am \
-DskipTests \
-Dcheckstyle.skip \
-Drat.skip \
package
```


附录 B 源码索引与笔记来源

本笔记基于第一周 days/notes、已完成实验、源码检查和对话中的问题重组。复习时优先定位以下文件：

主题	源码/笔记来源
第一周总览	learning-plan/week-01/README.md
每日手册与记录	learning-plan/week-01/days/day-01.md ... day-07.md; notes/day-01.md ... day-07.md
CLI/Runner/JCommander	ratls-examples/.../Runner.java; SubCommandBase.java; arithmetic/cli
Client 同步/异步	ratls-client/.../BlockingImpl.java; AsyncImpl.java; OrderedAsync.java; RaftClientImpl.java
请求对象	ratls-common/.../protocol/RaftClientRequest.java
Server 写链	ratls-server/.../impl/RaftServerImpl.java; LeaderStatImpl.java; ServerState.java
日志与 metadata	ratls-server/.../raftlog/RaftLogBase.java; LogProtoUtils.java
状态机	ratls-server/.../impl/StateMachineUpdater.java; ArithmeticStateMachine.java
三节点手册	learning-plan/week-01/days/day-06.md; notes/day-06.md
Git 同步脚本	learning-plan/week-01/scripts/windows/Sync-StudyBranch.ps1

最终记忆主线

POM 描述项目 -> Reactor 选择本次构建 -> 测试证明行为 -> Client 构造请求 -> Leader 建立日志顺序 -> 多数派提交 -> StateMachine apply -> Git commit 固化证据 -> Linux 同 commit 复验。